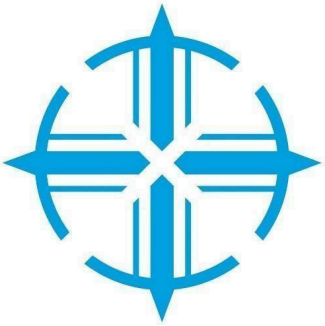
PONTIFICIA UNIVERSIDAD CATOLICA DEL ECUADOR



DESARROLLO EN PLATAFORMAS

T1S2: Proyecto: interacción JavaScript, validación dinámica y organización MVC para la plataforma de incidentes

Integrantes:

Ariel Israel Mejía Zambrano, Lenin Marcelo Moreno Guerrero, Ricardo Andrés Arias Barberan, Kevin Javier Montatixe Caiza

Docente:

Damián Aníbal Nicolalde Rodríguez

Quito - Ecuador

12 de Septiembre del 2026

Justificación técnica

El trabajo corresponde a la segunda versión de la plataforma de gestión y monitoreo de incidentes de ciberseguridad del Hospital Clínico San Rafael. No se construyó un proyecto nuevo: se partió del prototipo de la Unidad 1 y se conservaron su estructura semántica, su hoja de estilos y su identidad visual. El cambio es arquitectónico. Lo que era un documento HTML con los incidentes escritos a mano ahora es una aplicación de front-end que obtiene sus datos de una fuente externa y construye la vista durante la ejecución.

El código se organiza según una separación Modelo-Vista-Controlador, patrón que divide la representación interna de la información, su presentación y la gestión de la entrada del usuario (Krasner y Pope, 1988). Se implementa en tres archivos con fronteras claras. En `model.js`, la clase `IncidentModel` administra los datos y las reglas de negocio, y no accede al DOM en ningún punto. En `view.js`, la clase `IncidentView` se ocupa solo de la presentación: genera las filas, escribe los mensajes y actualiza los estados accesibles, sin saber de dónde vienen los datos ni qué reglas los validan. En `controller.js` se registran los escuchadores y se coordinan ambas capas, que nunca se comunican de forma directa. Como el modelo es el único que valida, la misma regla se aplica al abandonar un campo y al enviar el formulario, sin duplicar código.

El flujo sigue la secuencia JSON, modelo, controlador, vista y DOM. Los incidentes iniciales viven en `data/incidentes.json` y el modelo los solicita con la Fetch API. Conviene notar que `fetch()` no rechaza la promesa ante respuestas de error del servidor, sino que se resuelve con la propiedad `ok` en falso (MDN Web Docs, 2025a); por eso el modelo comprueba `response.ok` antes de interpretar el cuerpo. Después valida el esquema: que sea un arreglo, que cada elemento traiga sus seis campos como cadenas no vacías, que el código cumpla el patrón `INC-` más cuatro dígitos, que la severidad pertenezca al catálogo, que la fecha sea interpretable y que no haya códigos repetidos. La petición se limita a diez segundos con `AbortSignal.timeout()` para que un servidor sin respuesta no deje la interfaz esperando.

La renderización usa JavaScript moderno. La vista arma cada fila con `createElement`, la acumula en un `DocumentFragment` y la inserta con `replaceChildren`, de modo que el navegador ejecuta un solo reflujo en lugar de uno por fila. Los valores se escriben con `textContent` y nunca con `innerHTML`: es una decisión de seguridad coherente con el dominio del proyecto, porque `textContent` inserta el valor como texto plano y no como marcado interpretable (WHATWG, 2026). El código emplea además clases, `async/await`, plantillas literales, `Set`, encadenamiento opcional e `Intl.DateTimeFormat`.

La validación dinámica complementa la nativa en lugar de reemplazarla. Los atributos `required`, `pattern`, `minlength` y `maxlength` permanecen en el marcado y el controlador solo asigna `form.noValidate = true` al inicializarse; si JavaScript no está disponible, el navegador sigue aplicando la validación de restricciones (MDN Web Docs, 2025c). El relevo permite mensajes en lenguaje comprensible y reglas que el marcado no puede expresar: que el

código no esté ya registrado, que la fecha no sea futura, que las longitudes se midan tras recortar espacios y que el adjunto respete extensión y tamaño (MDN Web Docs, 2025b). Se valida en `blur` y `change`, y en `input` solo si el campo ya fue visitado, para no señalar un error mientras el usuario escribe el primer carácter. Al enviar, `preventDefault()` detiene el proceso si hay errores, se indica cuántos campos revisar y el foco pasa al primero.

La accesibilidad dinámica se apoya en ARIA porque los cambios ocurren sin recargar la página y, sin anuncio explícito, un lector de pantalla no los percibiría (W3C, 2023a). Cada control recibe `aria-invalid` según su estado y la vista suma el identificador del error a su `aria-describedby`, conservando la ayuda contextual de la Unidad 1. Los avisos informativos usan `role="status"` con `aria-live="polite"`, que espera una pausa del lector; los fallos usan `role="alert"`, que interrumpe. La tabla marca `aria-busy` durante la carga. Todo ello responde al principio de robustez de las WCAG 2.2 (W3C, 2023b).

El manejo de errores cubre el JSON inexistente, la caída de red, el vencimiento del tiempo de espera, el JSON mal formado, el esquema inválido y la lista vacía. En todos los casos se muestra un texto comprensible, se ofrece un botón para reintentar y el envío queda deshabilitado mientras el modelo no esté listo. El modelo es de solo lectura: los registros nuevos viven en memoria y el JSON nunca se reescribe, lo que corresponde al alcance de una demostración sin servidor. Por transparencia se indica que en esta entrega no se incorporaron jQuery ni una vista de detalle ampliada.

Evidencias de funcionamiento

Las capturas siguientes se obtuvieron ejecutando el proyecto desde un servidor local (`python -m http.server`), condición necesaria para que `fetch()` pueda leer el archivo JSON.

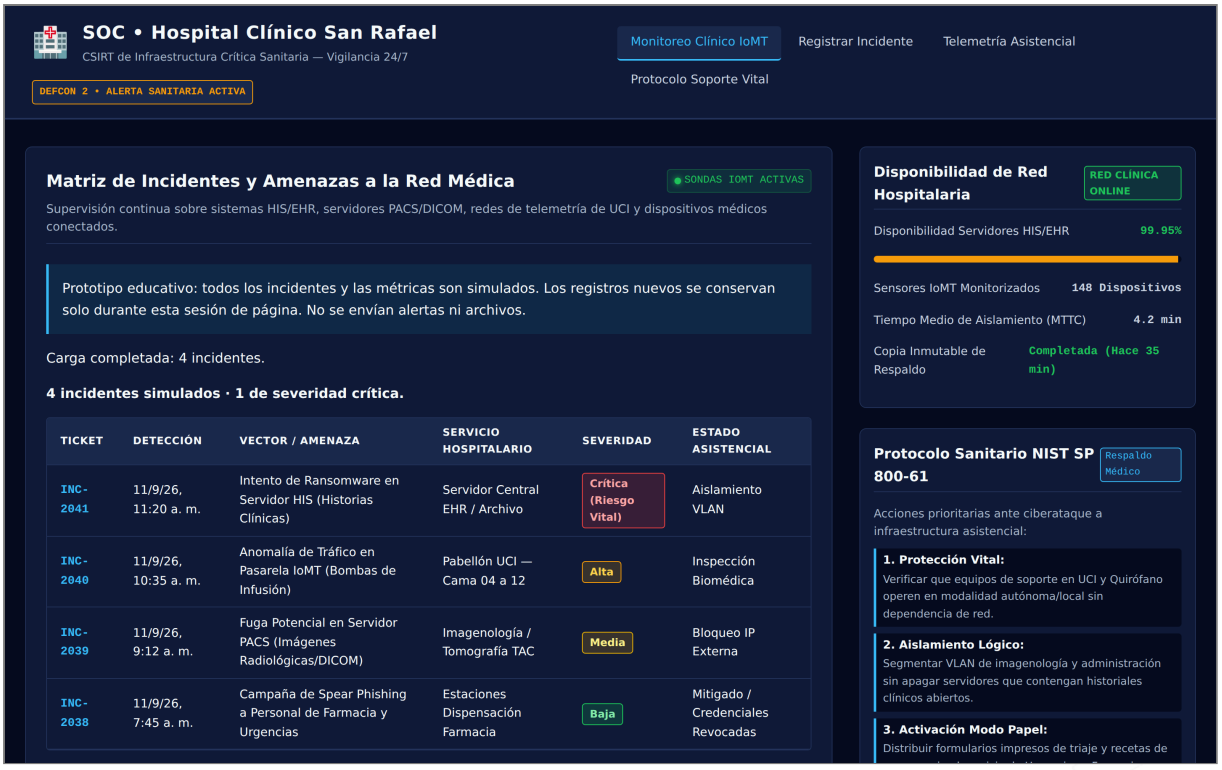


Figura 1. Interfaz principal en escritorio (1440 px). Demuestra la continuidad visual y estructural con la Unidad 1: se conservan la cabecera, la navegación, la distribución en cuadrícula, el panel lateral y la paleta cromática del prototipo original.

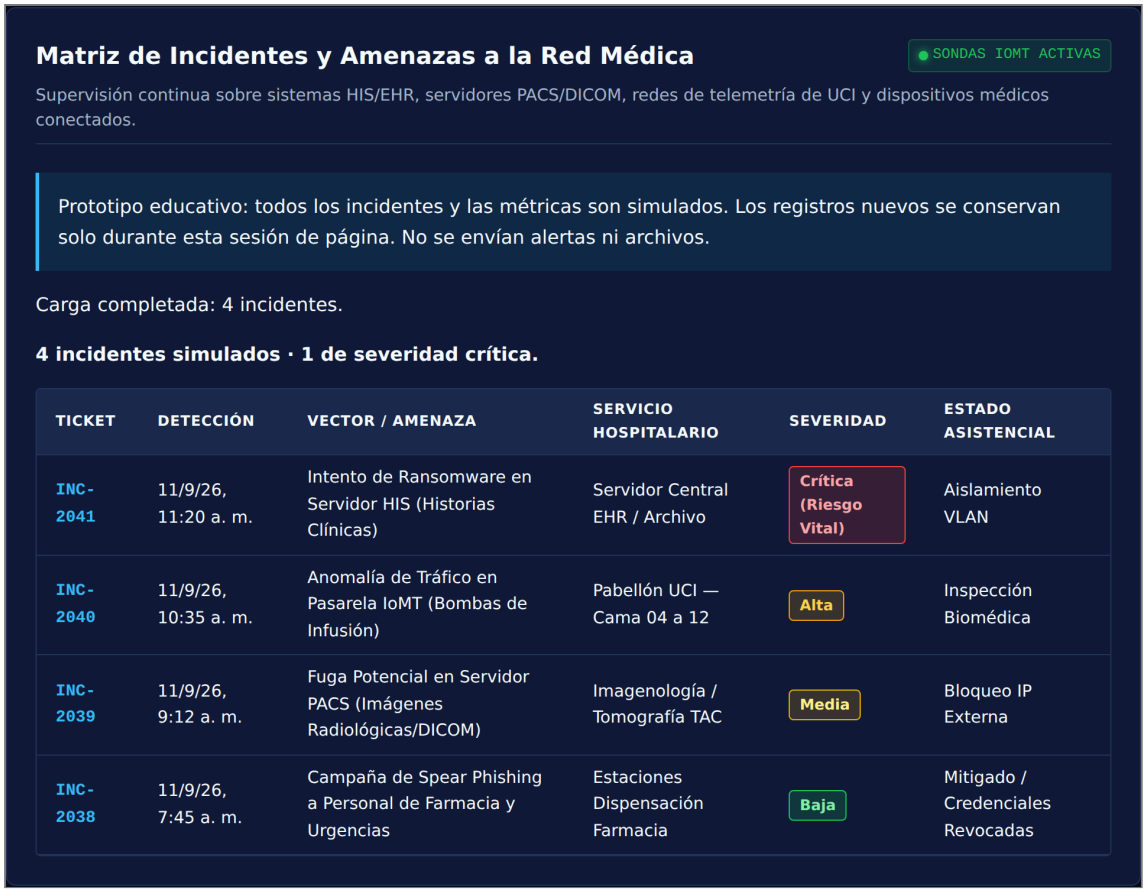


Figura 2. Renderizado dinámico. El cuerpo de la tabla llega vacío en el HTML y es construido por la vista a partir de los datos del modelo. El mensaje «Carga completada: 4 incidentes» y el contador de incidentes críticos son actualizaciones del DOM anunciadas mediante `aria-live`. Demuestra Fetch, JSON, DOM y manipulación dinámica.

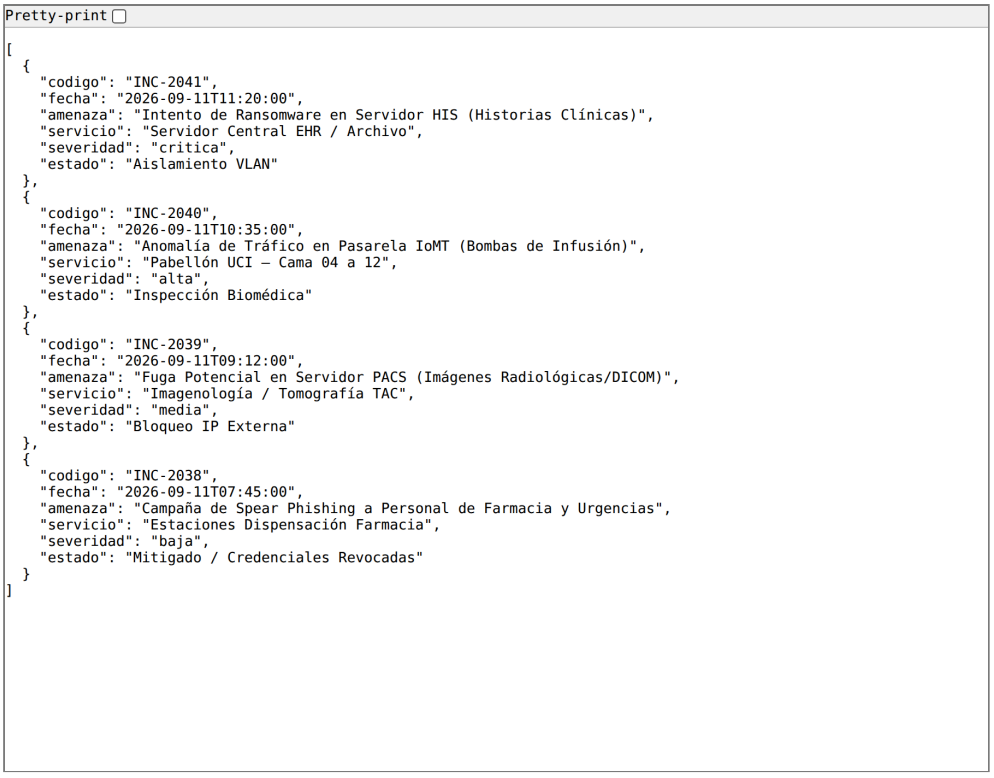


Figura 3. Fuente de datos `data/incidentes.json` servida por HTTP. Es el archivo que consume el modelo mediante Fetch y cuyo esquema se valida antes de aceptarlo. Demuestra el modelo local de solo lectura.



Figura 4. Formulario de triaje heredado de la Unidad 1, con sus tres agrupaciones, etiquetas asociadas y ayudas contextuales intactas. El botón de envío aparece habilitado porque el modelo ya terminó de cargar.



Figura 5. Validación dinámica con errores. Al intentar enviar el formulario vacío se previene el envío, se informa el número de campos por corregir, cada control recibe un mensaje propio y el foco se traslada al primer campo inválido. Demuestra eventos, validación dinámica y prevención del envío.

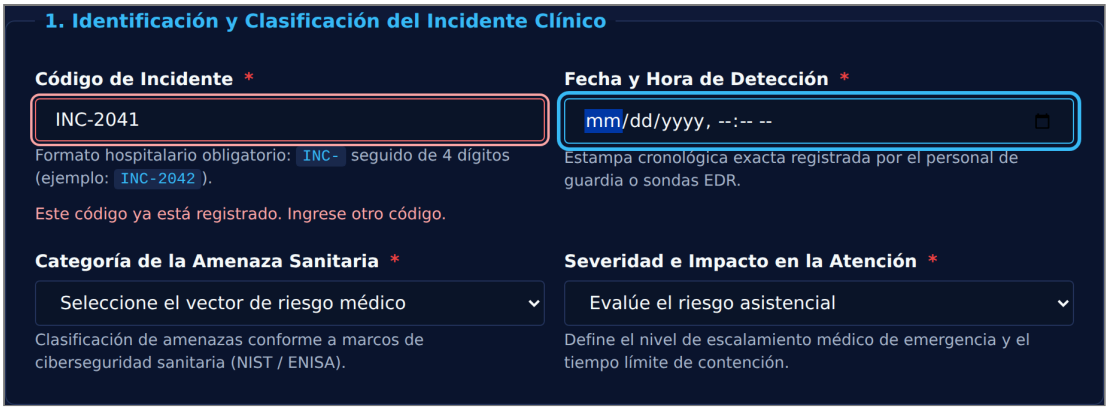


Figura 6. Regla que la validación nativa HTML5 no puede expresar: el código `INC-2041` se rechaza porque ya existe en el modelo. La comprobación se ejecuta al abandonar el campo y no requiere enviar el formulario. Demuestra la complementariedad entre la validación nativa y la validación por JavaScript.



Figura 7. DOM inspeccionado durante la ejecución. Se observa el `aria-invalid="true"` aplicado al control, el mensaje de error enlazado al campo a través de `aria-describedby` junto a la ayuda contextual original, y las regiones `role="status"` con `aria-live="polite"` que anuncian el resultado de la validación y de la carga. Demuestra accesibilidad dinámica real y no decorativa.

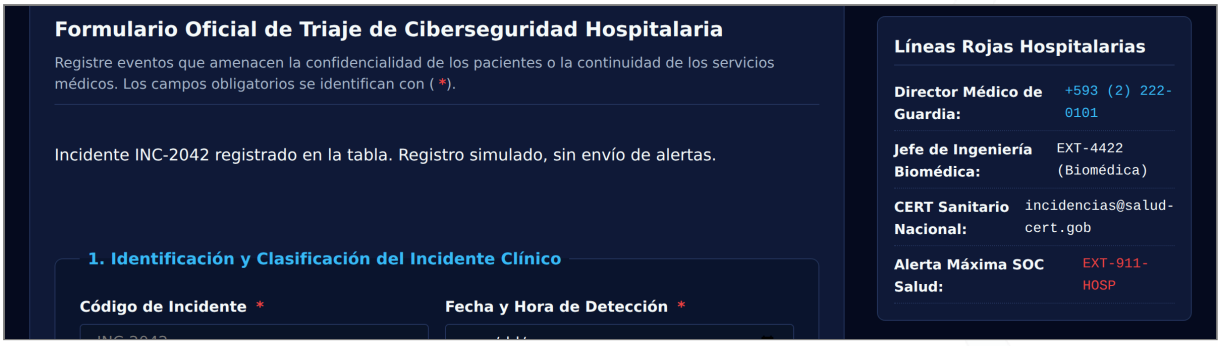


Figura 8. Validación correcta. Con todos los campos válidos y el consentimiento marcado, el controlador confirma el registro en la región accesible del formulario y aclara que se trata de un registro simulado.



Figura 9. Actualización del DOM sin recargar la página. El incidente `INC-2042` se incorporó al modelo en memoria, la vista lo volvió a renderizar en la primera posición y el contador pasó de cuatro a cinco incidentes. Demuestra la coordinación modelo-controlador-vista.



Figura 10. Manejo de errores. Con la petición del JSON respondiendo 404, el modelo lanza la excepción, el controlador la captura y la vista presenta un mensaje comprensible en una región `role="alert"`, ofrece el botón «Reintentar carga» y deja la tabla en un estado informativo en lugar de vacío. El botón de envío permanece deshabilitado.



Figura 11. Visualización responsiva en 390 px de ancho. Las Media Queries mobile-first de la Unidad 1 se conservan y los elementos incorporados en la Unidad 2 (avisos, mensajes de estado y botón de reintento) se integran en el flujo vertical sin romper la maquetación.

Matriz de cumplimiento

Requisito	Estado	Archivo	Evidencia y observaciones
JavaScript moderno	Implementado	`model.js, view.js, controller.js`	Clases, async/await, Set, plantillas literales, encadenamiento opcional, Intl.DateTimeFormat.
DOM	Implementado	`view.js`	Figuras 2 y 9. createElement, DocumentFragment, replaceChildren y textContent.
Eventos	Implementado	`controller.js`	Figuras 5 y 6. blur, input, change, submit, reset y click.
Validación dinámica	Implementado	`model.js (reglas), controller.js`	Figuras 5, 6 y 8. Complementa la validación nativa, que se conserva sin JavaScript.
Fetch API	Implementado	`model.js`	Figuras 2 y 10. Verificación de response.ok y tiempo de espera de 10 s.
JSON	Implementado	`data/incidentes.json`	Figura 3. Esquema validado antes de aceptar los datos.
ARIA	Implementado	`index.html, view.js`	Figura 7. aria-invalid, aria-describedby, aria-live, role status/alert, aria-busy.
jQuery	No implementado	—	No se incorporó en esta entrega. Identificado como trabajo pendiente.
MVC	Implementado	`js/model.js, js/view.js, js/controller.js`	El modelo no accede al DOM y la vista no conoce el origen de los datos.
Modelo local de solo lectura	Implementado	`model.js`	El JSON nunca se reescribe; los registros nuevos viven solo en memoria.
Manejo de errores	Implementado	`model.js, controller.js`	Figura 10. Red, HTTP, tiempo de espera, JSON inválido, lista vacía y código duplicado.
Vista de detalle ampliada	No implementado	—	La tabla muestra los campos disponibles en el modelo; no existe un panel de detalle independiente.
Responsividad	Conservado	`css/styles.css`	Figuras 1 y 11. Media Queries mobile-first heredadas de la Unidad 1.

Ejecución del proyecto

El proyecto debe abrirse mediante un servidor local. Al abrir `index.html` con doble clic, el navegador utiliza el protocolo `file://` y bloquea la petición de `fetch()` por política de mismo origen, con lo que la aplicación mostraría el mensaje de error de la Figura 10. Desde la carpeta raíz basta ejecutar `python -m http.server 8000` y visitar `http://localhost:8000`, o bien emplear la extensión Live Server.

Referencias Bibliográficas

Krasner, G. E., y Pope, S. T. (1988). A cookbook for using the model-view-controller user interface paradigm in Smalltalk-80. *Journal of Object-Oriented Programming, 1*(3), 26–49.

MDN Web Docs. (2025a). *Using the Fetch API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN Web Docs. (2025b). *Client-side form validation*. Mozilla. https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation

MDN Web Docs. (2025c). *Using HTML form validation and the Constraint Validation API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Constraint_validation

Pontificia Universidad Católica del Ecuador. (s. f.). *Desarrollo en plataformas* [Aula virtual]. EVA PUCE. https://eva.puce.edu.ec/EV-202502/course/view.php?id=71

W3C. (2023a). *Accessible Rich Internet Applications (WAI-ARIA) 1.2*. World Wide Web Consortium. https://www.w3.org/TR/wai-aria-1.2/

W3C. (2023b). *Web Content Accessibility Guidelines (WCAG) 2.2*. World Wide Web Consortium. https://www.w3.org/TR/WCAG22/

WHATWG. (2026). *DOM Standard*. Web Hypertext Application Technology Working Group. https://dom.spec.whatwg.org/